

Work Assignment

Phase 2 - Final

Learning outcomes

This assignment phase aims to help students to understand how to design and implement an **efficient** parallel version of the case study, using accelerators (e.g., GPUs) and/or using a distributed memory system.

Description

In this final assignment student have the freedom **to select one** of three alternative paths to improve the work in the previous phase: i) design and implement a new version for accelerators (GPU with CUDA, see annex B for specific help on the CUDA project); ii) design and implement a new version for distributed memory (C with MPI); iii) design a mix of OpenMP/MPI/CUDA.

Note: in the final assignment projects with data races or with the inadequate use of parallelism (e.g., inadequate usage of parallelism specification) will have a strong penalty.

The final assignment will be based on the same code as on the previous phase but with a different simulation (note: this is a simpler simulation with less hotspots). This version is available in a new repository: https://github.com/BMalaca/zpic_CompParalela_2aParte, but there is only small code changes from the previous phase (see annex A for a full list of changes).

One new request for this final assignment **is the selection and implementation a set of tests** for a solid performance assessment of the developed implementation. This may include the selection of different input sizes, different set of machines and an adequate scalability analysis.

The final assignment (report and presentation) should include a **discussion the key performance aspects** of the case study parallel implementation in order to justify the design decisions and their impact.

Evaluation

The final evaluation will consider:

- (i) **design and implementation** of the parallel version(s) of the case study **(20%)**;
- (ii) proposed **set of tests** for a scalability analysis **(10%)**;
- (iii) **understand and explain the performance results**, including models / metrics to justify the results **(25%)**;
- (iv) the **report quality** **(20%)**;
- (v) quality of the **oral presentation** **(25%)**

Groups, submission format and dates

The work assignment should be performed by the same student's groups from previous phases.

Submission rules are similar to previous assignments with minor changes (**in bold**):

- the submission must run on the Deucalion, **twostream** case study, **ppc = 500, no sorting** (see Annex A).
- the work must be submitted through the e-learning platform, compressed into a zip file that, when unzipped, should generate a directory whose name is the groups elements, e.g., a43000_pg54000, including:
 - a **6-page PDF final report** with the all relevant information concerning **all assignments**, using the same IEEE template of previous phases; annexes can be added beyond these 6 pages;
 - a subdirectory with all source code (please, do not submit executables, or other files);
 - a **Makefile** that generates and runs the executable: i) `make Makefile` should generate all required executables; ii) `make run` **should run the program by submitting execution to the cluster with sbatch, requesting the appropriate resources.**

Submission deadline: 23:59, 12-Jan-26.

The defence will be performed during the oral presentation from **14 to 16 of Jan.** The **oral presentation is mandatory.** Students should prepare a slide presentation of 7 min, which will be followed by a 10 min discussion.

Annex A – Code changes to the repository from the previous phase and how to scale the input size

1. in file `main.c`, line 33, change the input simulation configuration to `twostream.c`.

```
// Include Simulation parameters here
#include "input/twostream.c"
// #include "input/magnetized.c"
...
// #include "input/laser_particles.c"
```

2. The input size can be changed in the `twostream.c` configuration file (no other parameters should be changed in this file), ensure that the project submission uses the default `ppc` of 500.

```
// line 29 of twostream.c
int ppc = 500; // half of the number of particles to use
```

3. in file `particles.c`, line 579, disable particle sorting:

```
// Set default sorting frequency
spec -> n_sort = 100000; // place a high value here
```

Annex B – CUDA specific instructions

1. Sample generic CUDA Makefile:

```
CC = nvcc
CFLAGS = -O2 -g -std=c++11
# these are for search k20 GPU
#NCCFLAGS = -Wno-deprecated-gpu-targets -arch=sm_35 -O2 -g -std=c++11
NCCFLAGS = -Wno-deprecated-gpu-targets -arch=sm_80 -O2 -g -std=c++11
LDLFLAGS = -lm
LINKER      := nvcc

C_SOURCES   := $(wildcard *.c)
CU_SOURCES  := $(wildcard *.cu)
HEADERS     := $(wildcard *.h)
C_OBJS      := $(patsubst %.c, %.o, $(C_SOURCES))
CU_OBJS     := $(patsubst %.cu, %.o, $(CU_SOURCES))

BIN = zpic

$(BIN) : $(C_OBJS) $(CU_OBJS) $(HEADERS)
    $(LINKER) -o $(BIN) $(CU_OBJS) $(C_OBJS) $(LDLFLAGS) $(LIBS)

$(C_OBJS) : $(C_SOURCES) $(HEADERS)
    $(CC) -c $(C_SOURCES) $(CFLAGS)

$(CU_OBJS) : $(CU_SOURCES) $(HEADERS)
    $(CC) -c $(CU_SOURCES) $(NCCFLAGS)

clean:
    rm -f $(BIN) *.o
```

2. CUDA float3 name clash

CUDA defines a `float3` type that clashes with the `zpic` definition on the `zpic.h` header file. Change `zpic` file to use CUDA `float3` definition.

```
// file zpic.h
/**
 * @brief Three component vector
 *
 */
#include<cuda_runtime.h>
/*typedef struct Float3 {
    float x;    ///< x vector component
    float y;    ///< y vector component
    float z;    ///< z vector component
} float3;*/
```

3. Suggestion for correct C/C++ linkage

This example shows how to call a GPU function from the base `zpic` project. This solves the problem of compiling the `zpic` project with C linkage, but CUDA modules are always compiled with C++ linkage.

```
// file main.c (this is compiled with C linkage)

...

extern void showIterNumber(int i);

int main (int argc, const char * argv[]) {

    // Initialize simulation
    t_simulation sim;
    sim_init( &sim );

    ...

    for (n=0,t=0.0; t<=sim.tmax; n++, t=n*sim.dt) {
        //printf("n = %i, t = %f\n",n,t);
        showIterNumber(n);
    }

    ...

// file iter.cu (this is compiled with C++ linkage)

#include <stdio.h>
#include <cuda.h>

__global__ void myKernel(int *iter) {
    printf("iteration %d\n",*iter);
}

// declares this function with C linkage
extern "C" void showIterNumber(int i) {

    int *di;

    cudaMalloc ((void**) &di, sizeof(int) );
    cudaMemcpy (di, &i, sizeof(int), cudaMemcpyHostToDevice);

    myKernel<<<1,1>>>>(di);

    cudaFree ((void**) di );
}
```

4. Suggestion for GPU batch job

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --gpus=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=32
#SBATCH --time=04:00
#SBATCH --partition normal-a100-40
#SBATCH --gpus=1
#SBATCH --account=f202500010hpcvlabuminhog

ml CUDA

srun -nl ./zpic
```